

Міністерство освіти і науки України
Львівський національний університет імені Івана Франка
Факультет електроніки та комп'ютерних технологій

Звіт
про виконання лабораторної роботи №1
з курсу «Машинне навчання»
Лінійна регресія, оптимізація та статичний аналіз

Виконав:
студент групи ФЕІ-44
Гілета І. І.

Перевірив:
доцент Сінькевич О. О.

Львів 2026

Мета роботи: зрозуміти архітектуру лінійної регресії, навчитися самостійно реалізовувати алгоритми оптимізації, обирати інформативні ознаки, обчислювати метрики якості та перевіряти статистичні припущення моделі

Завдання:

Для досягнення поставленої мети необхідно виконати комплексний аналіз даних на прикладі набору Ames Housing Dataset, що включає первинне дослідження, відбір ознак на основі кореляції та правильну попередню обробку з дотриманням правил розбиття вибірки, кодування категорій та масштабування.

Після підготовки даних потрібно математично реалізувати функцію прогнозування, функцію втрат та обчислення градієнтів, використовуючи виключно векторизовані матричні операції бібліотеки numpy без використання стандартних циклів.

Наступним важливим кроком є розробка та порівняння трьох алгоритмів оптимізації: повного (Batch), стохастичного (SGD) та міні-пакетного (Mini-batch) градієнтного спуску, з обов'язковою візуалізацією процесу мінімізації функції втрат.

Після успішного навчання моделі вимагається провести оцінку її якості за допомогою коефіцієнта детермінації на тренувальній, валідаційній та тестовій вибірках. Заключна частина роботи передбачає перевірку ключових статистичних припущень лінійної регресії шляхом візуального аналізу залишків моделі, а також виконання додаткового експерименту для дослідження геометричного впливу масштабування ознак на стабільність роботи градієнтного спуску.

ТЕОРЕТИЧНІ ВІДОМОСТІ

Лінійна регресія та функція втрат Лінійна регресія - це базова модель машинного навчання, яка описує залежність цільової змінної від однієї або кількох незалежних ознак за допомогою лінійної функції. У матричному вигляді модель записується як $y = Xw + b$, де X - матриця ознак, w - вектор

ваг, а b - зміщення. Для оцінки точності передбачень найчастіше використовується функція втрат Mean Squared Error, яка обчислює середнє значення квадратів різниць між реальними та прогнозованими значеннями.

Оптимізація та градієнтний спуск Мінімізація функції втрат здійснюється за допомогою алгоритму градієнтного спуску (Gradient Descent), який ітеративно оновлює ваги моделі в напрямку антиградієнта. Розмір кроку на кожній ітерації визначається параметром швидкості навчання (Learning Rate, α). Залежно від об'єму даних розрізняють:

- **Batch GD (повний)**: обчислює градієнт на основі всієї навчальної вибірки (стабільна збіжність, але довго).
- **SGD (стохастичний)**: використовує лише один випадковий приклад для кроку (швидко, але сильні коливання).
- **Mini-batch GD**: обробляє дані невеликими пакетами (наприклад, по 32/64 приклади) - оптимальний компроміс.

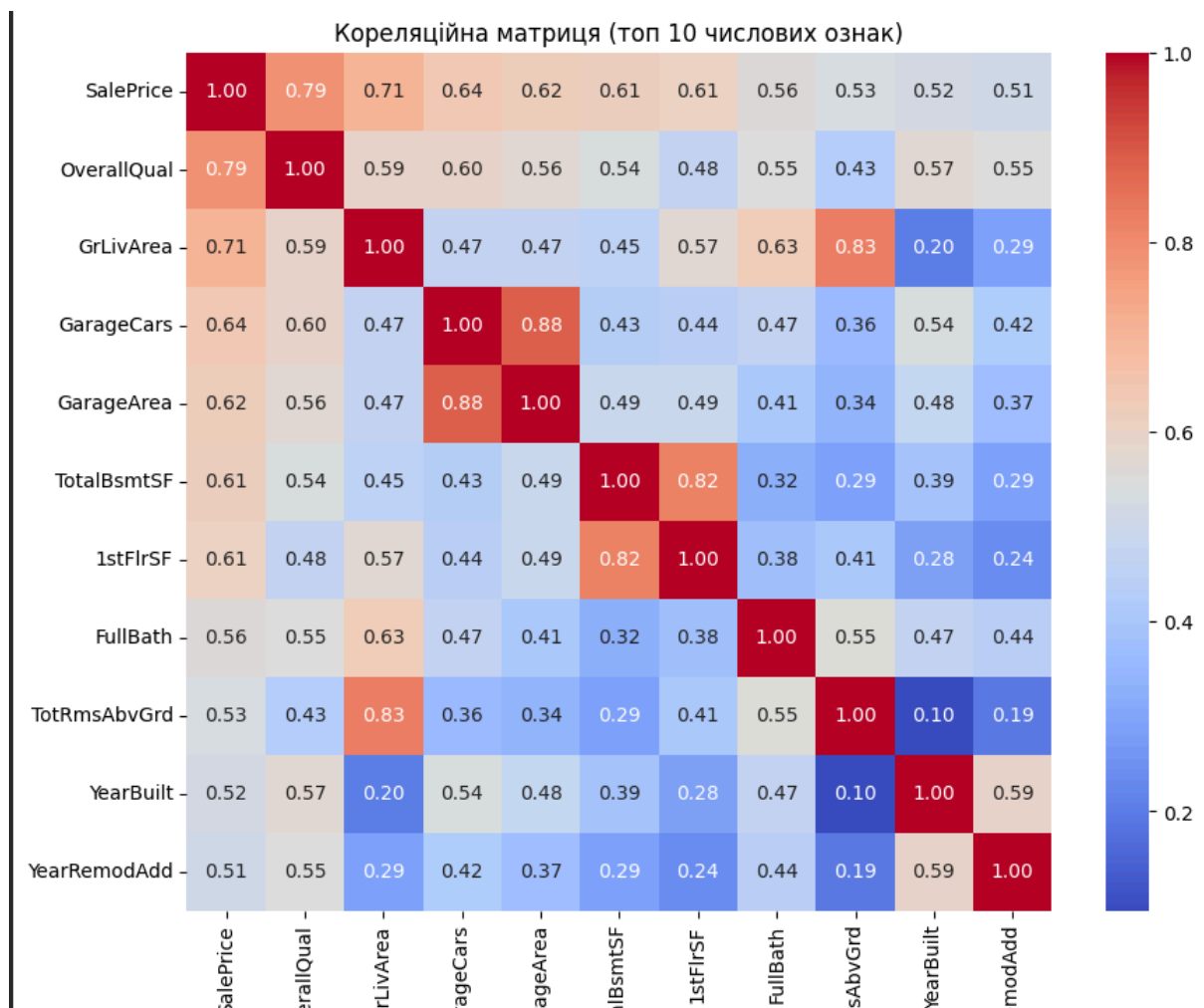
Попередня обробка даних та оцінка якості Для швидкості градієнтного спуску критично важливим є масштабування. Щоб уникнути витоку даних, масштабування параметрів застосовується виключно після розбиття датасету. Якість моделі оцінюється через R^2 , який показує, яку частку дисперсії цільової змінної пояснює модель.

Статистичні припущення моделі До ключових припущень належать: наявність лінійної залежності, нормальний розподіл залишків, гомоскедастичність та відсутність мультиколінеарності.

Хід роботи:

Етап 1. Дослідження даних (EDA), відбір ознак та попередня обробка

Для виконання роботи я обрав середовище Google Colab та завантажив Ames Housing Dataset за допомогою `fetch_openml`. Після очищення даних від ознак, що містять більше 50% пропусків, я побудував кореляційну матрицю.



Для моделі було відібрано числові ознаки з найвищою кореляцією з SalePrice (OverallQual, GrLivArea, GarageCars, TotalBsmtSF, 1stFlrSF, YearBuilt), а також категоріальні ознаки (Neighborhood, BldgType, HouseStyle, KitchenQual). Далі здійснено розбиття вибірки (Train - 60%, Validation - 20%, Test - 20%). Для категорій застосовано OneHotEncoder, для числових - StandardScaler.

Відповіді на теоретичні питання (Етап 1):

- **Чому розбиваємо дані до масштабування та що таке Data Leakage?** Дані розбиваються до нормалізації, щоб уникнути витоку даних. Якщо застосувати масштабування на всьому датасеті відразу, середнє значення та дисперсія розрахуються з урахуванням тестових даних, що штучно завищить точність моделі. Параметри скейлера мають обчислюватися виключно на Train.
- **Як обробити нові категорії на валідації/тесті?** Використано OneHotEncoder з параметром `handle_unknown='ignore'`. Якщо

з'явиться нова категорія, енкодер просто заповнить відповідні координати нулями, і програма продовжить роботу.

Етап 2. Реалізація Лінійної Регресії засобами numpy

Створено клас NumpyLinearRegression для обчислення прогнозу, функції втрат (MSE) та градієнтів. Замість циклів for використано векторизовані матричні операції (наприклад, $X.T @ error$).

```
class NumpyLinearRegression:
    def __init__(self, learning_rate=0.01, epochs=1000, method='batch', batch_size=32):
        self.lr = learning_rate
        self.epochs = epochs
        self.method = method
        self.batch_size = batch_size
        self.w = None
        self.b = None
        self.loss_history = []

    def fit(self, X, y):
        m, n = X.shape
        self.w = np.zeros(n) # Ініціалізація wag нулями
        self.b = 0           # Ініціалізація bias
        y = np.array(y)
        self.loss_history = []

        for epoch in range(self.epochs):
            if self.method == 'batch':
                # Прогнозування ( $X * w + b$ )
                y_hat = X @ self.w + self.b
                error = y_hat - y

                # Векторизоване обчислення градієнтів (без for-циклів)
                dw = (2/m) * (X.T @ error)
                db = (2/m) * np.sum(error)

                # Оновлення параметрів
                self.w -= self.lr * dw
                self.b -= self.lr * db

                # Збереження Cost Function (MSE)
                self.loss_history.append(np.mean(error**2))

            elif self.method == 'sgd':
                # Стохастичний градієнтний спуск (по 1 екземпляру)
                indices = np.random.permutation(m)
```

Етап 3. Градієнтний спуск: три варіанти оптимізації

Розроблено три методи: Batch GD, SGD та Mini-batch GD. Навчивши моделі впродовж 150 епох, побудовано порівняльний графік.



Відповіді на теоретичні питання (Етап 3):

- **Який метод збігається найстабільніше?** Найстабільнішим є Batch GD, оскільки антиградієнт обчислюється точно по всій вибірці, і функція втрат плавно спадає.
- **Який метод має найбільші коливання?** SGD має значні стрибки, оскільки крок розраховується лише по одному випадковому прикладу, алгоритм сильно відхиляється.
- **Вплив розміру batch:** Розмір батчу (напр. 64 у Mini-batch GD) є золотою серединою. Він згладжує коливання, але потребує менше ресурсів на ітерацію, пришвидшуючи навчання.
- **Вплив Learning Rate:** Правильний вибір гарантує збіжність. Якщо крок занадто великий - алгоритм перестрибне мінімум і розійдеться. Якщо занадто малий - навчання буде занадто довгим.

Етап 4. Оцінка якості та коефіцієнт детермінації (R^2)

Реалізовано розрахунок R^2 та перевірено навчену модель.

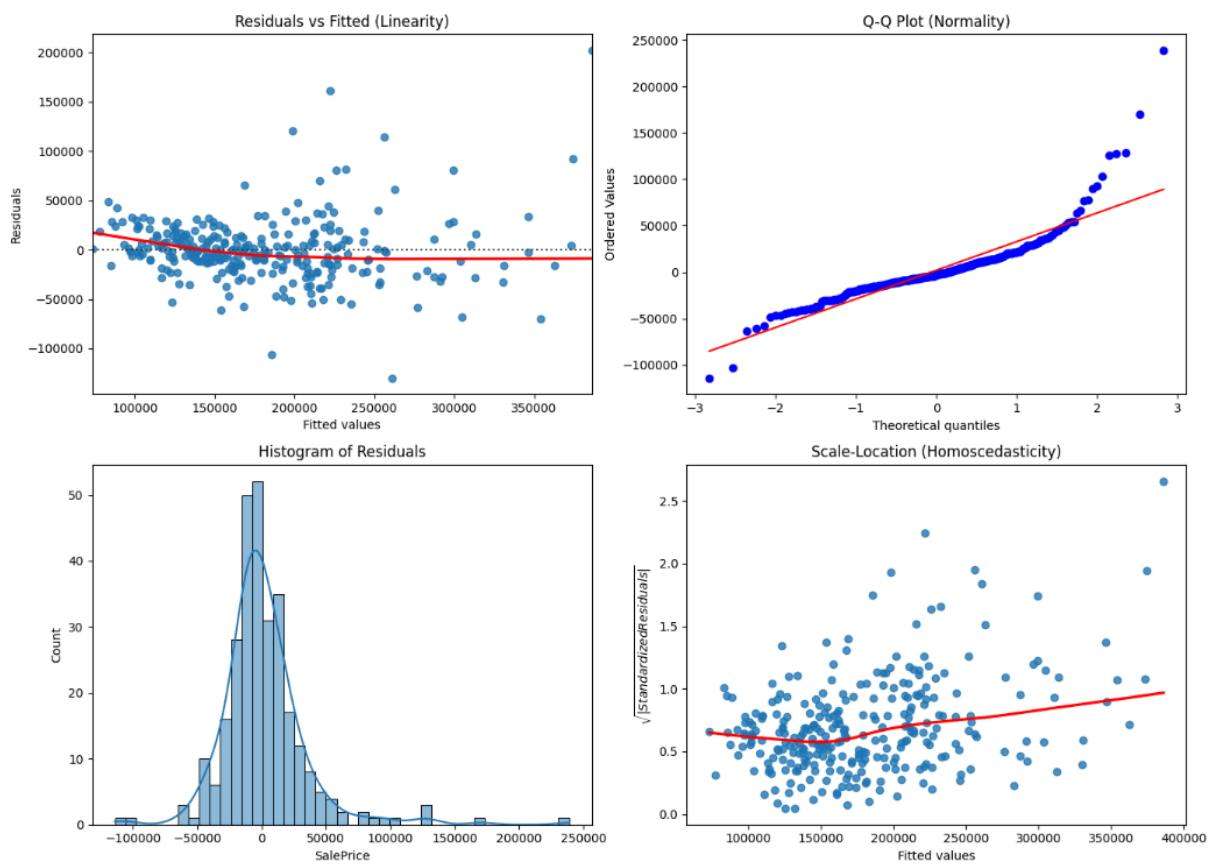
```
R2 Train: 0.8242
R2 Val:   0.8084
R2 Test:  0.8178
```

Відповіді на теоретичні питання (Етап 4):

- **Статистичний зміст R^2 :** Метрика показує, яку частку дисперсії цільової змінної пояснює модель.
- **Чи може R^2 бути від'ємним?** Так, якщо сума квадратів помилок нашої моделі перевищує помилку константної моделі (яка завжди прогнозує середнє).
- **Train vs Test (стан моделі):** Якщо показник R^2 на Test близький до Train і не є суттєво нижчим, це свідчить, що модель чудово узагальнює дані і не має критичного перенавчання.

Етап 5. Перевірка статистичних припущень

Обчислено залишки для вибірки та побудовано діагностичні графіки за допомогою seaborn і matplotlib. Також розраховано VIF для перевірки мультиколінеарності.



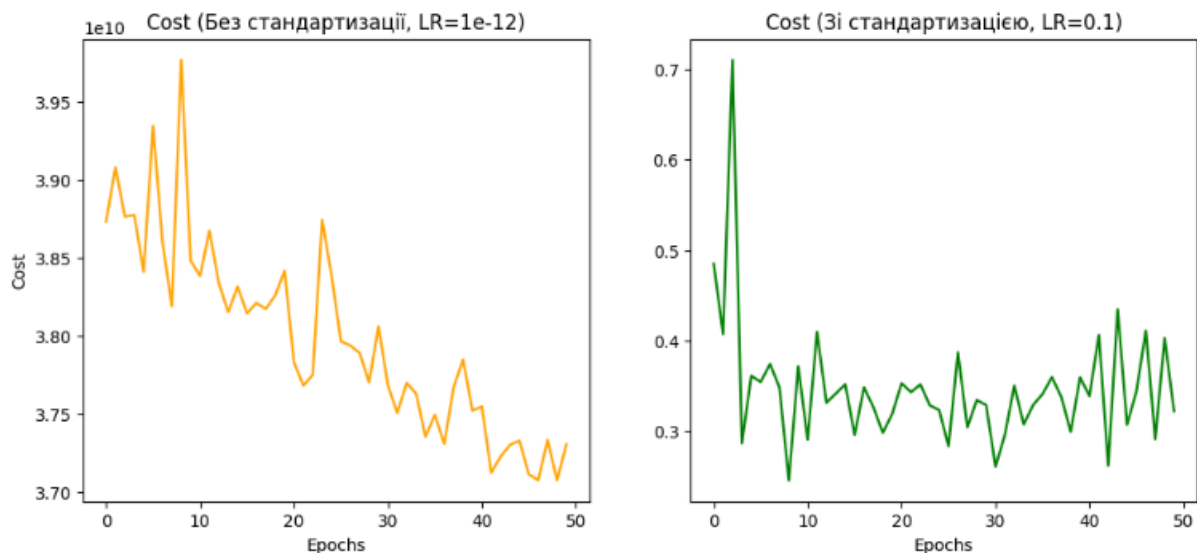
Висновки щодо припущень моделі (Етап 5):

1. **Лінійність:** Виконується частково. Лінія тренду загалом лежить біля нуля, але відхиляється для дорогих будинків.

2. **Нормальність залишків:** Порушується. Q-Q Plot відхиляється від діагоналі, а гістограма має важкий правий хвіст через ексклюзивні дорогі будинки.
3. **Гомоскедастичність:** Порушується. На графіку Scale-Location видно ефект "конуса" або "воронки" (гетероскедастичність) - дисперсія помилок зростає зі зростанням ціни.
4. **Мультиколінеарність:** Присутня помірна мультиколінеарність (особливо між показниками площі та роком побудови), що підтверджується значеннями VIF.

Етап 6. Завдання для глибокого розуміння (Експеримент А)

Провів експеримент із впливом масштабування, запустивши Mini-batch GD на стандартизованих та немасштабованих даних.



Відповідь на теоретичне питання (Експеримент А): Геометричне пояснення: Без стандартизації ознаки мають різні порядки величин. Геометрично поверхня функції втрат набуває форми витягнутого еліпса. Антиградієнт відбивається від стінок, і алгоритм осцилює, тому довелося значно зменшити Learning Rate. Після стандартизації поверхня стала симетричною (сферичною), що дозволило взяти великий крок ($\alpha=0.1$) і миттєво знайти оптимум.

ВИСНОВОК

Під час виконання лабораторної роботи розроблено модель лінійної регресії за допомогою векторизованих матричних операцій бібліотеки `numpy`. Коректна попередня підготовка даних (масштабування та кодування строго після розбиття датасету) дозволила повністю уникнути витоку інформації.

Порівняння алгоритмів оптимізації довело, що Mini-batch GD є найкращим компромісом між швидкістю та стабільністю. Модель показала достатньо високу здатність до узагальнення, коефіцієнт детермінації R^2 підтвердив адекватну точність без надмірного перенавчання.

Аналіз залишків підтвердив загальну працездатність моделі, хоча й виявив очікувану для ринку нерухомості гетероскедастичність (збільшення розкиду помилок для дорогих об'єктів) та відхилення від нормальності, що свідчить про доцільність логарифмування цільової змінної у майбутньому.

Фінальний експеримент практично довів критичну роль стандартизації: приведення ознак до спільного масштабу геометрично вирівнює поверхню функції втрат, що дозволяє значно збільшити крок навчання та гарантує швидку й безпроблемну збіжність градієнтного спуску.